

Lab 4: SIMD Processing (AVX2) - Detailed Study Guide

This comprehensively expanded guide explains Single Instruction, Multiple Data (SIMD) processing using AVX2 intrinsics. We will break down vector operations, memory layout best practices, conditional logic without loops, and solve typical exam questions step by step.

1. What is SIMD and AVX2?

Instead of adding an array of 8 floats sequentially taking 8 separate CPU clock cycles (Scalar execution), AVX2 (Advanced Vector Extensions 2) provides massive 256-bit hardware registers. Since a standard `float` is 32-bits, $256 / 32 = 8$. You can pack 8 floats into a single register and execute one addition instruction that adds all 8 floats simultaneously.

The Rule of Inclusion

Always compile your code with the `-mavx2` and `-O3` flags: `g++ -O3 -mavx2 program.cpp` Include the library header: `#include <immintrin.h>`

Intrinsics Data Types

- `__m256`: A custom type holding **eight** 32-bit floats.
 - `__m256d`: Holds **four** 64-bit doubles.
 - `__m256i`: Holds 256-bits of integer data (usually eight 32-bit ints).
-

2. Essential SIMD Toolkit & Step-by-Step Skeleton

When writing SIMD code, memory alignment is your biggest concern. If your memory pointers align perfectly with 32-byte chunks (e.g. `aligned_alloc`), you can use the super-fast `_mm256_load_ps` instruction. However, most exam questions provide a raw `float*` without guaranteed alignment.

Rule of Thumb: Always default to unaligned loads (`_mm256_loadu_ps`) unless explicitly asked inside an exam question to optimize using `aligned_alloc`.

The Core Loop Pattern

Every SIMD array question follows the exact same "Modulo Pattern". When your array size isn't perfectly divisible by 8, you have leftover elements.

How to structure it:

1. A `for` loop that steps by 8: `i += 8`
2. Stop when the *next* block of 8 wouldn't fit: `i <= size - 8`
3. A "Tail loop" (Sequential) that steps by 1 to finish the remainder: `i++`

```
// Goal: C[i] = A[i] * B[i] + A[i]
void compute_array(float* a, float* b, float* c, int size) {
```

```

int i = 0;

// 1. The SIMD Bulk Loop
for (; i <= size - 8; i += 8) {
    __m256 vec_a = _mm256_loadu_ps(&a[i]);           // Load 8 floats from a
    __m256 vec_b = _mm256_loadu_ps(&b[i]);           // Load 8 floats from b

    // Fused Multiply-Add computes (A*B)+C natively. Very fast.
    __m256 vec_c = _mm256_fmadd_ps(vec_a, vec_b, vec_a);

    _mm256_storeu_ps(&c[i], vec_c);                 // Store 8 floats into c
}

// 2. The Scalar Tail Loop (for the leftovers)
for (; i < size; i++) {
    c[i] = a[i] * b[i] + a[i];
}
}

```

3. How to Execute "If-Statements" inside SIMD (Masking/Blending)

Because SIMD executes multiple data elements at exactly the same time, it **cannot** take diverging execution branches (what if element 1 is true but element 2 is false?).

The SIMD Concept: Masking & Blending To conditionally do something:

1. **Compute the True condition:** Compute the answer assuming the IF statement is TRUE for ALL elements.
2. **Compute the False condition:** (Or just use zeros/original values).
3. **Generate a Mask:** Use a compare intrinsic to build a bitmask (1s where true, 0s where false).
4. **Blend the Vectors:** Combine the True Vector and False Vector together based on the Mask.

Example: Rectified Linear Unit (ReLU) — sets negatives to 0.

```

// Scalar: output[i] = (input[i] > 0) ? input[i] : 0.0f;
__m256 vec_in = _mm256_loadu_ps(&input[i]);
__m256 zeros = _mm256_setzero_ps(); // Create a vector full of 0.0f

// 1. Create bit mask. Syntax: _mm256_cmp_ps(A, B, Comparison_Macro)
// _CMP_GT_OQ translates to Greater-Than
__m256 mask = _mm256_cmp_ps(vec_in, zeros, _CMP_GT_OQ);

// 2. Blend: Pick element from 'vec_in' if mask is 1, else pick from 'zeros'
__m256 result = _mm256_blendv_ps(zeros, vec_in, mask);
_mm256_storeu_ps(&output[i], result);

```

4. Exam Cheat Sheet: Adapting SIMD Code

If the question says...	What intrinsic/pattern to use...	Code Snippet
"Add a constant scalar value XX to every element"	Broadcast XX into a vector first, then use <code>add_ps</code> .	<pre>__m256 val = _mm256_set1_ps(X); vec_res = _mm256_add_ps(vec_arr, val);</pre>
"Memory is guaranteed to be 32-byte aligned"	Use the faster aligned load/store instructions.	<pre>_mm256_load_ps(ptr) and _mm256_store_ps(ptr, vec) (No 'u')</pre>
"Find the overall sum / dot product"	Accumulate in a vector inside the loop, do a Horizontal Sum at the end.	<code>_mm_hadd_ps(...)</code> (See Dot Product question below for exact steps)
"Clamp maximum values to XX "	Broadcast XX to a vector, then use SIMD min operator.	<pre>__m256 max_val = _mm256_set1_ps(X); vec = _mm256_min_ps(vec, max_val);</pre>
"Apply operation only if element > 0 "	Use Masking and Blending. NO <code>if</code> statements!	<pre>__m256 mask = _mm256_cmp_ps(vec, zero, _CMP_GT_OQ); _mm256_blendv_ps(zero, vec, mask);</pre>

5. Multi-Layer Exam Preparation: Solved Mock Questions

Below are step-by-step solutions to typical SIMD AVX2 exam questions.

Question A: Normalizing a Dataset (Mathematical Broadcasts)

Problem Statement: You have an array of signals `input`. The task is to subtract a `baseline` scalar value from every element, and if the final result is less than 0, clamp it to 0. Write a SIMD function `void normalize(float* input, float* output, int size, float baseline)` to do this efficiently.

Step-by-step Solution:

- Preparation:** We need a vector that only contains `baseline` across all 8 slots to subtract from the input. We use `_mm256_set1_ps()`.
- Subtraction:** We use `_mm256_sub_ps()`.
- Clamping (Max):** Clamping to 0 is just finding the maximum between the element and 0. `_mm256_max_ps()` is perfect here.

```
void normalize(float* input, float* output, int size, float baseline) {
    int i = 0;
    __m256 vec_base = _mm256_set1_ps(baseline); // Broadcast baseline
    __m256 vec_zero = _mm256_setzero_ps();      // Vector of 0.0f

    for (; i <= size - 8; i += 8) {
```

```

    __m256 vec_in = _mm256_loadu_ps(&input[i]);

    // Subtract baseline
    __m256 vec_sub = _mm256_sub_ps(vec_in, vec_base);

    // Clamp to 0 by picking max(x, 0)
    __m256 vec_result = _mm256_max_ps(vec_sub, vec_zero);

    _mm256_storeu_ps(&output[i], vec_result);
}

// Tail loop
for(; i < size; i++) {
    float val = input[i] - baseline;
    output[i] = (val > 0.0f) ? val : 0.0f;
}
}

```

Question B: The Horizontal Sum (Dot Product)

Problem Statement: You are computing the dot product of two enormous arrays: $C = \sum (A[i] \times B[i])$. The standard SIMD loop results in an `__m256` vector holding 8 partial sums. Explain, with pseudo-code, the technique to accumulate these 8 slots into a single scalar float.

Step-by-step Solution:

- Iterative Multiplication:** You write the loop using `_mm256_fmadd_ps` accumulating into an `__m256 sum_vec`. When the loop finishes, `sum_vec` stores `[S0, S1, S2, S3, S4, S5, S6, S7]`.
- Split the Vector:** Cut the 256-bit vector into two 128-bit blocks (4 floats each).
 - `__m128 high = _mm256_extractf128_ps(sum_vec, 1);`
 - `__m128 low = _mm256_castps256_ps128(sum_vec);`
- Sum Halves:** `__m128 sum128 = _mm_add_ps(high, low);` (We now have 4 sums)
- Horizontal Additions:** The `_mm_hadd_ps(v, v)` takes `[s0, s1, s2, s3]` and structurally alters it to `[s0+s1, s2+s3, s0+s1, s2+s3]`.
 - Apply it twice: `sum128 = _mm_hadd_ps(sum128, sum128);` (down to 2 unique sums) `sum128 = _mm_hadd_ps(sum128, sum128);` (down to 1 unique sum repeated)
- Extract:** `float final_scalar = _mm_cvtss_f32(sum128);`

Question C: The 1D Convolution Filter Box

Problem Statement: Perform a 1D convolution on `input` using a 3-element filter `kernel`. Generate `output[i] = input[i]*k[0] + input[i+1]*k[1] + input[i+2]*k[2]`.

Step-by-step Solution: Unlike typical parallel arrays, we only load `kernel[0]`, `kernel[1]`, `kernel[2]` into constant broadcast vectors. And instead of hopping `j` across the loop chunk, we iterate across the kernel.

```

void convolve_simd_3(float* input, float* kernel, float* output, int len) {
    int i = 0;
    int out_len = len - 3 + 1; // 1D Conv output bound

```

```
// Broadcast the 3 kernel weights
__m256 k0 = _mm256_set1_ps(kernel[0]);
__m256 k1 = _mm256_set1_ps(kernel[1]);
__m256 k2 = _mm256_set1_ps(kernel[2]);

for (; i <= out_len - 8; i += 8) {
    // We load the 8 starting slots.
    __m256 in0 = _mm256_loadu_ps(&input[i]);
    // We load shifted by 1 index!
    __m256 in1 = _mm256_loadu_ps(&input[i + 1]);
    // We load shifted by 2 indexes!
    __m256 in2 = _mm256_loadu_ps(&input[i + 2]);

    // Accumulate FMA: sum = in0*k0
    __m256 sum = _mm256_mul_ps(in0, k0);
    sum = _mm256_fmadd_ps(in1, k1, sum); // in1*k1 + sum
    sum = _mm256_fmadd_ps(in2, k2, sum); // in2*k2 + sum

    _mm256_storeu_ps(&output[i], sum);
}

// Process remainder sequentially...
}
```